

Practica 6: refactorizaciones

Raúl Rivero, Ángel González

1. Aplicación a estudiar

La aplicación gestiona conductores y los transportes que realizan. Cada conductor tiene un sueldo base de 700 euros y un extra por transporte. En el código inicial, la clase Conductor calcula todos los extras con un switch sobre CategoriaTransporte, mientras que Transporte guarda campos que no siempre se usan (ton o personas según la categoría).

2. Métricas iniciales

El cálculo de WMC y CCoG esta anotado también como comentario en el código fuente de cada clase.

Clase	WMC	WMCn	Ccog	Ccogn	CBO	DIT	NOC
CategoriaTransporte	0	0	0	0	0	0	0
Transporte	8	1.6	4	0.8	2	0	0
Conductor	18	2	9	1	2	0	0
GestionTransportes	6	2	4	1.33	1	0	0
GestionTransportesGUI	18	9	34	17	4	0	0

Las clases correspondientes al CBO están escritas en el código entregado.

3. Refactorizaciones aplicadas

1. Replace Conditional with Polymorphism: se sustituye el campo CategoriaTransporte cat por una jerarquía de transportes: TransportePersonas, TransporteMercancias y TransporteMercanciasPeligrosas.
2. Extract Method: el switch de Conductor.sueldo() se elimina. Cada transporte sabe calcular su propio extra.
3. Extract Method: la GUI extrae la creación de transportes y el cálculo de mejores conductores a métodos privados.
4. Rename Class: gestionTransportes pasa a GestionTransportes, siguiendo la convención Java para nombres de clases.

4. Métricas finales

Clase	WMC	WMCn	Ccog	Ccogn	CBO	DIT	NOC
CategoriaTransporte	0	0	0	0	0	0	0
Transporte	5	1.25	1	0.25	1	0	0
TransportePersonas	6	2	2	0.67	1	1	0
TransporteMercancias	4	1.33	1	0.33	1	1	0
TransporteMP	2	1	0	0	0	2	0
Conductor	15	1.5	3	0.3	2	0	0
GestionTransportes	6	2	4	1.33	1	0	0
GestionTransportesGUI	20	5	27	6.75	7	0	0

5. Análisis de la mejora

La mejora principal se refleja claramente en Conductor: WMC baja de 18 a 15 y CCog baja de 9 a 3. Esto ocurre porque Conductor deja de conocer las reglas concretas de cada categoría y solo suma el sueldo de cada transporte. La clase queda más estable ante cambios: si aparece un nuevo tipo de transporte, se añade una clase nueva sin modificar el método sueldo().

La complejidad total se reparte entre clases pequeñas. DIT y NOC aumentan porque ahora existe una jerarquía de herencia, pero ese aumento es razonable: representa una decisión de diseño que sustituye condicionales por polimorfismo.

6. Diagrama final

